

Jenkins Pipeline Project Tasks - Step by Step Solutions

Task 1: Simple Echo Pipeline

1. Create a new pipeline job in Jenkins named ****EchoPipeline****.
2. Use the following Declarative pipeline script:

```
...  
pipeline {  
  agent any  
  stages {  
    stage('Echo') {  
      steps {  
        echo 'Hello, Jenkins Pipeline!'  
      }  
    }  
  }  
}
```

3. Run the job → Check console output → You should see ``Hello, Jenkins Pipeline!``.

Task 2: Multi-Stage Pipeline

1. Create a pipeline job called ****BuildTestPipeline****.
2. Use the following script:

```
...  
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps { echo 'Building the project...' }  
    }  
    stage('Test') {  
      steps { echo 'Running tests...' }  
    }  
    stage('Deploy') {  
      steps { echo 'Deploying application...' }  
    }  
  }  
}
```

3. Run the job → Stage view will show Build, Test, and Deploy stages.

Task 3: Workspace File Handling

1. Create a pipeline job called ****FilePipeline****.
2. Script:

```
...
```

```

pipeline {
  agent any
  stages {
    stage('Create File') {
      steps {
        writeFile file: 'pipeline.txt', text: 'Created by Jenkins Pipeline'
      }
    }
    stage('Show File') {
      steps {
        powershell 'Get-Content pipeline.txt' // use 'cat' for Linux
      }
    }
  }
}

```

3. Run job → Console output will display file content.

Task 4: Parameterized Pipeline

1. Create a pipeline job called **UserPipeline**.
2. Add a **String Parameter** named `USERNAME`.
3. Script:

```

'''
pipeline {
  agent any
  parameters {
    string(name: 'USERNAME', defaultValue: 'User', description: 'Enter your name')
  }
  stages {
    stage('Greet') {
      steps {
        echo "Hello, ${params.USERNAME}!"
      }
    }
  }
}
'''

```

4. Run builds with different USERNAME values.

Task 5: Scheduled Pipeline

1. Create a job called **ScheduledPipeline**.
2. Configure build triggers → Add `H/5 * * * *` (every 5 minutes).
3. Script:

```

'''
pipeline {
  agent any
  triggers {
    cron('H/5 * * * *')
  }
  stages {

```

```

stage('Show Time') {
  steps {
    powershell 'Get-Date' // use 'date' for Linux
  }
}
}
}
}
}
}

```

4. Jenkins will trigger the job automatically every 5 minutes.

Task 6: Git-Integrated Pipeline

1. Create a job called ****GitPipeline****.
2. Configure it with your GitHub repo URL.
3. Script:

```

...

pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/your-repo.git'
      }
    }
    stage('Build') {
      steps {
        echo 'Compiling code...'
      }
    }
    stage('Commit Info') {
      steps {
        powershell 'git log -1 --pretty=%B' // use 'sh' on Linux
      }
    }
  }
}
}
}
}
}

```

4. Run → Jenkins will pull code, simulate build, and show last commit message.